



www.isea.gov.in



Cryptography

Sugata Gangopadhyay

Department of Computer Science and
Engineering

Indian Institute of Technology
Roorkee



Week 4

Module 1

- Differential attack

Differential Attack

- Differential cryptanalysis is a chosen ciphertext attack.
- We assume that an attacker has a large number of tuples (x, x^*, y, y^*) where $x' = x \oplus x^*$ and $y' = y \oplus y^*$.
- The plaintext elements x, x^* are encrypted by using the same unknown key, K , yielding the ciphertexts y and y^* , respectively.
- For each candidate key, we compute the values of certain state bits, and determine if their XORs has a certain value (namely, the most likely value for the given XOR).
- Whenever it does, we increment a counter corresponding to the particular candidate key.

Differential properties of S -boxes

- Let $\pi_S: \{0, 1\}^m \rightarrow \{0, 1\}^n$ be an S -box. Consider an (ordered) pair of bitstrings of length m , say (x, x^*) . We say that the input XOR of the S -box is $x \oplus x^*$ and the output XOR is $\pi_S(x) \oplus \pi_S(x^*)$. Note that the output XOR is a bitstring of length n .
- For any $x' \in \{0, 1\}^m$, define $\Delta(x')$ to consist of all ordered pairs (x, x^*) having input XOR equal to x' .
- The set $\Delta(x')$ contains 2^m pairs, and
$$\Delta(x') = \{(x, x \oplus x') : x \in \{0, 1\}^m\}$$

S-box: Differential Profile

- Consider the following S -box:

x	0	1	2	3	4	5	6	7
$\pi_S(x)$	E	4	D	1	2	F	B	8

x	8	9	A	B	C	D	E	F
$\pi_S(x)$	3	A	6	C	5	9	0	7

0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111



Difference distribution

- For the above S -box suppose we consider the input XOR $x' = 1011$.
- Then the elements of $\Delta(1011)$ are

(0000, 1011)	(1000, 0011)
(0001, 1010)	(1001, 0010)
(0010, 1001)	(1010, 0001)
(0011, 1000)	(1011, 0000)
(0100, 1111)	(1100, 0111)
(0101, 1110)	(1101, 0110)
(0110, 1101)	(1110, 0101)
(0111, 1100)	(1111, 0100)

Difference distribution

- The distribution of the outputs when the input difference is 1011:

0000	0001	0010	0011	0100	0101	0110	0111
0	0	8	0	0	2	0	2

1000	1001	1010	1011	1100	1101	1110	1111
0	0	0	0	0	2	0	2

- The above distribution shows that only 5 out of 16 possible output XORs occur.

Difference distribution

x	x^*	y	y^*	y'
0000	1011	1110	1100	0010
0001	1010	0100	0110	0010
0010	1001	1101	1010	0111
0011	1000	0001	0011	0010
0100	1111	0010	0111	0101
0101	1110	1111	0000	1111
0110	1101	1011	1001	0010
0111	1100	1000	0101	1101
1000	0011	0011	0001	0010
1001	0010	1010	1101	0111
1010	0001	0110	0100	0010
1011	0000	1100	1110	0010
1100	0111	0101	1000	1101
1101	0110	1001	1011	0010
1110	0101	0000	1111	1111
1111	0100	0111	0010	0101

x	0	1	2	3	4	5	6	7
$\pi_S(x)$	E	4	D	1	2	F	B	8

x	8	9	A	B	C	D	E	F
$\pi_S(x)$	3	A	6	C	5	9	0	7

	0000	0001	0010	0011	0100	0101	0110	0111
0	0	0	8	0	0	2	0	2
1000	1000	1001	1010	1011	1100	1101	1110	1111
	0	0	0	0	0	2	0	2

Differential profile of an S -box

- For a bitstring x' of length m and a bitstring y' of length m , define
$$N_D(x', y') = |\{ (x, x^*) \in \Delta(x') : \pi_S(x) \oplus \pi_S(x^*) = y' \}|$$
- $N_D(x', y')$ counts the number of pairs with input XOR equal to x' that also have output XOR equal to y' (for a given S -box).
- The table in the next slide contains the tabulation of all the values of $N_D(a', b')$ where $a', b' \in \{0, 1\}^4$

Difference distribution table: values of $N_D(a', b')$

a'	b'															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	2	0	0	0	2	0	2	4	0	4	2	0	0
2	0	0	0	2	0	6	2	2	0	2	0	0	0	0	2	0
3	0	0	2	0	2	0	0	0	0	4	2	0	2	0	0	4
4	0	0	0	2	0	0	6	0	0	2	0	4	2	0	0	0
5	0	4	0	0	0	2	2	0	0	0	4	0	2	0	0	2
6	0	0	0	4	0	4	0	0	0	0	0	0	2	2	2	2
7	0	0	2	2	2	0	2	0	0	2	2	0	0	0	0	4
8	0	0	0	0	0	0	2	2	0	0	0	4	0	4	2	2
9	0	2	0	0	2	0	0	4	2	0	2	2	2	0	0	0
A	0	2	2	0	0	0	0	0	6	0	0	2	0	0	4	0
B	0	0	8	0	0	2	0	2	0	0	0	0	0	2	0	2
C	0	2	0	0	2	2	2	0	0	0	0	2	0	6	0	0
D	0	4	0	0	0	0	0	4	2	0	2	0	2	0	2	0
E	0	0	2	4	2	0	0	0	6	0	0	0	0	0	2	0
F	0	2	0	0	6	0	0	0	0	4	0	2	0	0	2	0

Propagation ratio

- The propagation ratio $R_p(a', b')$ for the differential (a', b') is defined as follows:

$$R_p(a', b') = \frac{N_D(a', b')}{2^m}$$

- $R_p(a', b')$ can be interpreted as a conditional probability:

$$\Pr[\text{output xor} = b' \mid \text{input xor} = a'] = R_p(a', b')$$

Week 4

Module 2

- Differential attack

Differential attack

- Suppose that we find propagation ratios for differentials in consecutive rounds of the SPN, such that the input XOR of a differential in any round is the same as the (permuted) output XORs of the differentials in the previous round.
- Then these differentials can be combined to form a differential trail.
- We make the assumption that the various propagation ratios in a differential trail are independent (an assumption that may not be mathematically valid).
- This assumption allows us to multiply the propagation ratios of the differentials in order to obtain the propagation ratio of the differential trail.

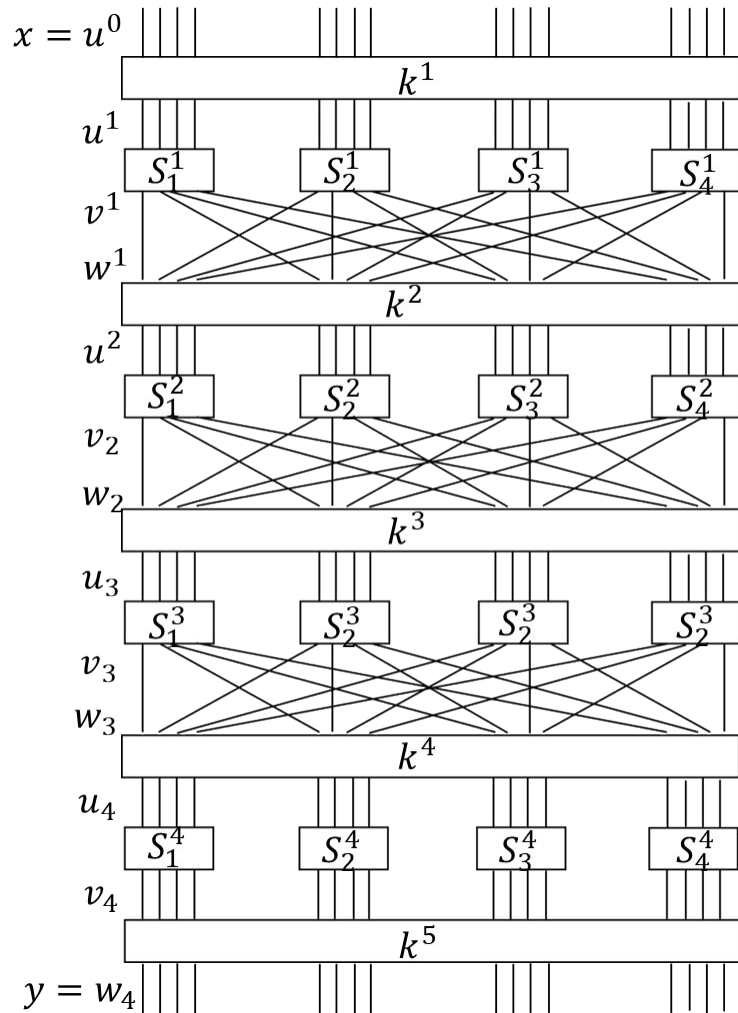
Substitution-Permutation Network

- In this lecture we describe a Substitution-Permutation Network (SPN) that we cryptanalyze using differential attack in the following lecture.
- In an SPN the block length $n = \ell m$. Each block is divided into subblocks of size ℓ .
- While executing a round
 - Each subblock is operated by an $\ell \times \ell$ S-box, $\pi_S: \{0,1\}^\ell \rightarrow \{0,1\}^\ell$
 - The individual bits of the block are permuted by a permutation $\pi_P: \{1, \dots, \ell m\} \rightarrow \{1, \dots, \ell m\}$.
- The total number of rounds is denoted by $\mathcal{N}$.

Substitution-Permutation Network

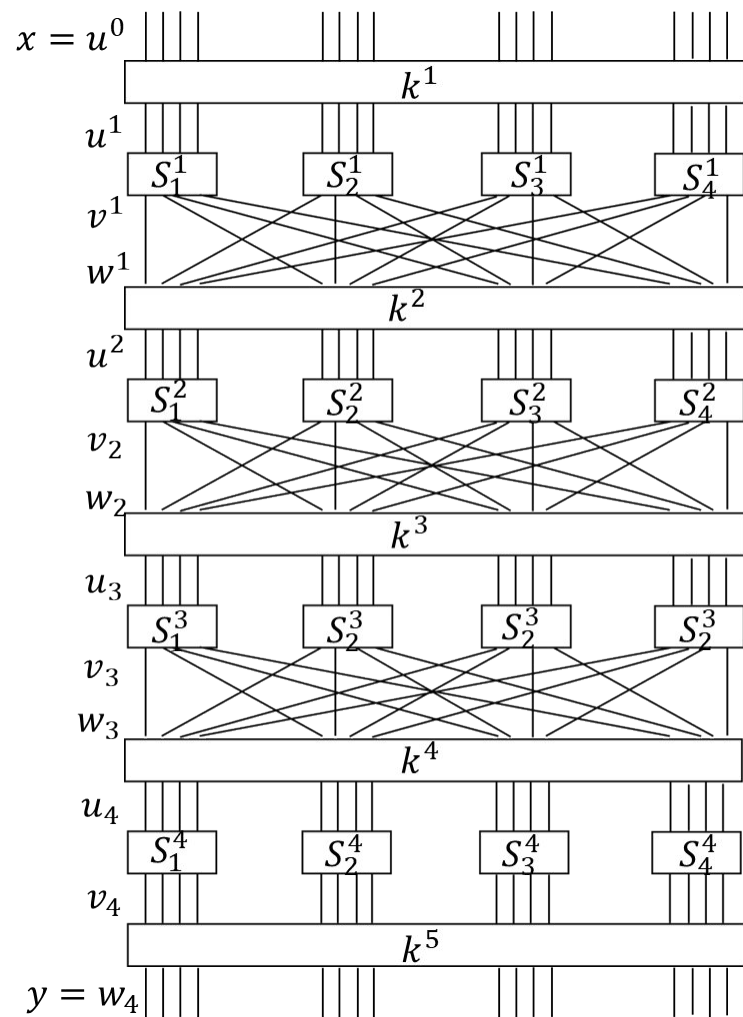
- The input block is: $x = x_{\langle 1 \rangle} \cdots x_{\langle m \rangle}$ where $x_{\langle i \rangle} = x_{(i-1)\ell+1} \cdots x_{(i-1)\ell+\ell}$.
- The r -th round key is K^r where $K^r = K_1^r \cdots K_{\ell m}^r$.
- r -th round input : $u^r = u_{\langle 1 \rangle}^r \cdots u_{\langle m \rangle}^r$ where $u_{\langle i \rangle}^r = u_{(i-1)\ell+1}^r \cdots u_{(i-1)\ell+\ell}^r$
- After substitution in the r -th round $v_{\langle i \rangle}^r = \pi_S(u_{\langle i \rangle}^r)$: $v^r = v_{\langle 1 \rangle}^r \cdots v_{\langle m \rangle}^r$
where $v_{\langle i \rangle}^r = v_{(i-1)\ell+1}^r \cdots v_{(i-1)\ell+\ell}^r$
- After permutation the output: $w^r = v_{\pi_P(1)}^r \cdots v_{\pi_P(\ell m)}^r$
- In the last round the permutation is not applied.

SPN: an example



- Key $K = (K_1, \dots, K_{32})$
- For $1 \leq r \leq 5$, K^r is a vector consisting of 16 consecutive bits of K .
- $K^r = (K_{4r-3}, \dots, K_{4r+12})$
 $= (K_1^r, \dots, K_{16}^r) = (K_{\langle 1 \rangle}^r, K_{\langle 2 \rangle}^r, K_{\langle 3 \rangle}^r, K_{\langle 4 \rangle}^r)$
- π_S and π_P are described in the next two slides.

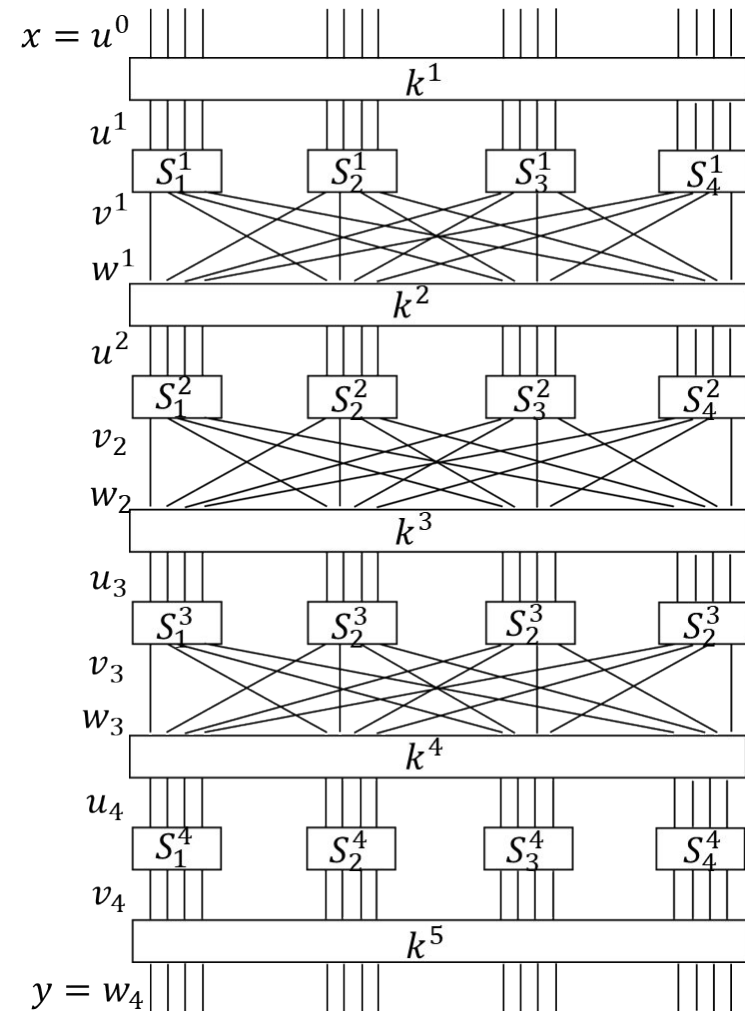
Difference distribution



x	0	1	2	3	4	5	6	7
$\pi_S(x)$	E	4	D	1	2	F	B	8

x	8	9	A	B	C	D	E	F
$\pi_S(x)$	3	A	6	C	5	9	0	7

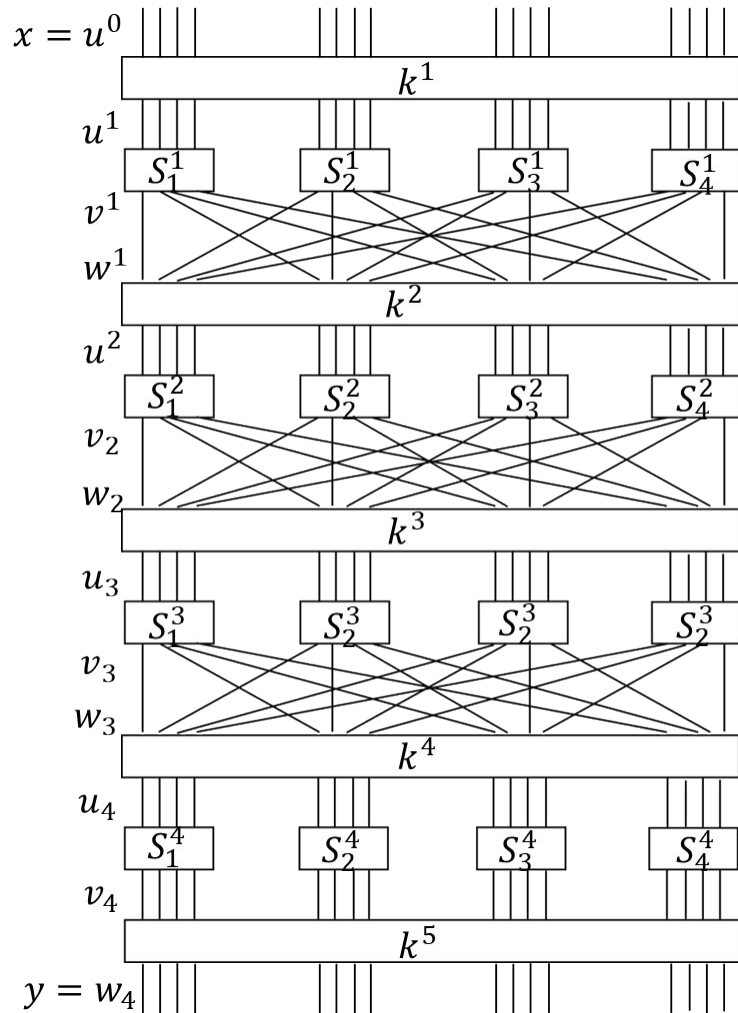
Difference distribution



x	1	2	3	4	5	6	7	8
$\pi_P(x)$	1	5	9	13	2	6	10	14

x	9	10	11	12	13	14	15	16
$\pi_S(x)$	3	7	11	15	4	8	12	16

Differential attack



x	0	1	2	3	4	5	6	7
π_S	E	4	D	1	2	F	B	8
x	8	9	A	B	C	D	E	F
π_S	3	A	6	C	5	9	0	7

- In S_2^1 , $R_p(1011, 0010) = \frac{1}{2}$
- In S_3^2 , $R_p(0100, 0110) = \frac{3}{8}$
- In S_2^3 , $R_p(0010, 0101) = \frac{3}{8}$
- In S_3^3 , $R_p(0010, 0101) = \frac{3}{8}$

These differentials can be combined to form the following differential trail:

$$R_p(0000101100000000, 0000010101010000) = \frac{1}{2} \times \left(\frac{3}{8}\right)^3 = \frac{27}{1024}.$$

Differential Attack

- $x' = 0000101100000000 \Rightarrow (v^3)' = 0000010101010000$
with probability $\frac{27}{1024}$.
- $(v^3)' = 0000010101010000 \Rightarrow (u^4)' = 00000110000000110$
with probability $\frac{27}{1024}$.
- $x' = 0000101100000000 \Rightarrow (u^4)' = 00000110000000110$
with probability $\frac{27}{1024}$.
- Therefore,

$$R_p = (0000101100000000, 0000010101010000) = \frac{1}{2} \times \left(\frac{3}{8}\right)^3 = \frac{27}{1024}.$$

Differential Attack Algorithm

Input: $\mathcal{T}, T, \pi_S^{-1}$

Output: *maxkey*

for $(L_1, L_2) \leftarrow (0, 0)$ **to** (F, F) **do**

Count $(L_1, L_2) \leftarrow 0$;

end

for $(x, y, x^*, y^*) \in \mathcal{T}$ **do**

if $y_{\langle 1 \rangle} = (y_{\langle 1 \rangle})^*$ **and** $y_{\langle 3 \rangle} = (y_{\langle 3 \rangle})^*$ **then**

for $(L_1, L_2) \leftarrow (0, 0)$ **to** (F, F) **do**

$v_{\langle 2 \rangle}^4 \leftarrow L_1 \oplus y_{\langle 2 \rangle}$;

$v_{\langle 4 \rangle}^4 \leftarrow L_2 \oplus y_{\langle 4 \rangle}$;

$u_{\langle 2 \rangle}^4 \leftarrow \pi_S^{-1}(v_{\langle 2 \rangle}^4)$;

$u_{\langle 4 \rangle}^4 \leftarrow \pi_S^{-1}(v_{\langle 4 \rangle}^4)$;

$(v_{\langle 2 \rangle}^4)^* \leftarrow L_1 \oplus (y_{\langle 2 \rangle})^*$;

$(v_{\langle 4 \rangle}^4)^* \leftarrow L_2 \oplus (y_{\langle 4 \rangle})^*$;

$(u_{\langle 2 \rangle}^4)^* \leftarrow \pi_S^{-1}((v_{\langle 2 \rangle}^4)^*)$;

$(u_{\langle 4 \rangle}^4)^* \leftarrow \pi_S^{-1}((v_{\langle 4 \rangle}^4)^*)$;

$(u_{\langle 2 \rangle}^4)' \leftarrow u_{\langle 2 \rangle}^4 \oplus (u_{\langle 2 \rangle}^4)^*$;

$(u_{\langle 4 \rangle}^4)' \leftarrow u_{\langle 4 \rangle}^4 \oplus (u_{\langle 4 \rangle}^4)^*$;

if $(u_{\langle 2 \rangle}^4)' = 0110$ **and** $(u_{\langle 4 \rangle}^4)' = 0110$ **then**

Count $[L_1, L_2] \leftarrow \text{Count}[L_1, L_2] + 1$;

end

end

end

end

max $\leftarrow -1$;

for $(L_1, L_2) \leftarrow (0, 0)$ **to** (F, F) **do**

if *Count* $[L_1, L_2] > \text{max}$ **then**

max $\leftarrow \text{Count}[L_1, L_2]$;

maxkey $\leftarrow (L_1, L_2)$;

end

return *maxkey*

end

Week 4

Module 3

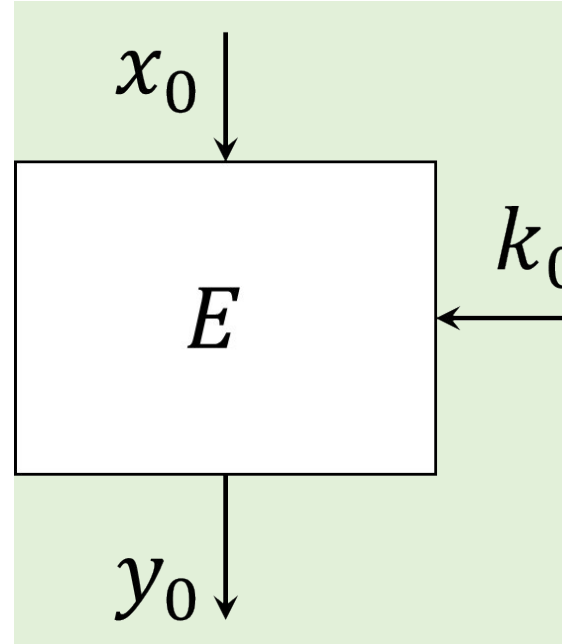
Objective: ***TMTO Part – 1***

To learn

- The time-memory tradeoff attack on a block cipher

Chosen plaintext attack on a block cipher

- We assume that the adversary knows the plaintext-ciphertext pair (x_0, y_0) , but does not know the key k_0 .
- The goal of the adversary is to find the secret key k_0



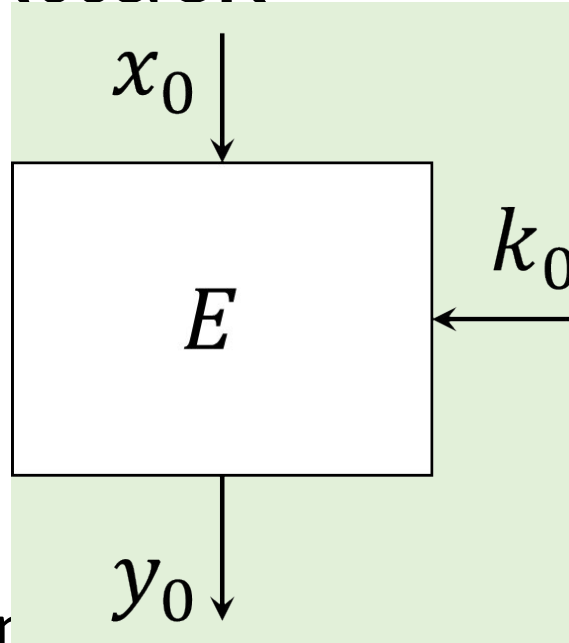
Schematic diagram of a block cipher

Exhaustive Search Attack

- Precompute the following table

k_1	$E(x_0, k_1) = y_1$
k_2	$E(x_0, k_2) = y_2$
$\vdots$	$\vdots$
k_{2^m}	$E(x_0, k_{2^m}) = y_{2^m}$

- In the online phase search for y_0 in the rightmost column of Table 4.
- If $y_j = y_0$ then $k_j = k_0$ is the secret key.



Exhaustive Search Attack

(The foundation of time-memory tradeoff)



- The running time complexity of the pre-computation phase is $P = O(2^m)$
- The memory complexity is $M = O(2^m)$
- The running time complexity in the online phase is
$$T = O(\log_2 2^m) = O(m)$$
if the list is sorted.
- If we do not store the preprocessing table, then $M = O(1)$ and $T = 2^m$.

Time-memory tradeoff

- In the late seventies, after acceptance of DES as the encryption standard, Martin Hellman proposed a time-memory trade-off attack on DES.

(Hellman, M.E.: A cryptanalytic time-memory trade-off. IEEE Trans.

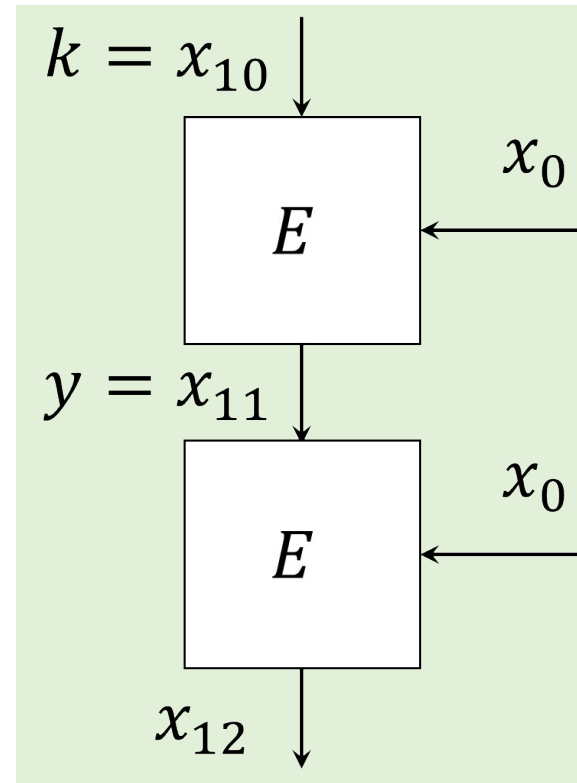
Information Theory 26(4), 401 – 406 (1980))

Time-memory tradeoff

- We saw that if we have access to memory of $O(2^m)$, we can search for the key in $O(1)$ time. Conversely, if we allow computation of order $O(2^m)$, it is possible to mount an attack with $O(1)$ memory.
- Hellman proposed a middle path between these two extreme cases and demonstrated a practical attack on DES.

TMTO: Hellman's Approach

- Hellman considers block ciphers for which $\mathcal{P} = \mathcal{K} = \mathcal{C} = \mathbb{F}_2^n$.
- Chosen plaintext attack: the attacker can choose a plaintext, x say, and obtain the ciphertext $y = E(x, k)$ during the actual operational phase of the cipher without the knowledge of the key $k \in \mathcal{K}$ in use.



Assuming $n = m$, $f: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ defined by $f(x_{ij}) = E(x_0, x_{ij}) = x_{i(j+1)}$

Preparing for the Precomputation Phase

- $\mathcal{P} = \mathcal{C} = \mathcal{K} = \mathbb{F}_2^n$. Keyspace size $N = 2^m$.
- Block cipher $E: \mathcal{P} \times \mathcal{K} \rightarrow \mathcal{C}$
- $f: \mathcal{K} \rightarrow \mathcal{C}$ is $f(x_{ij}) = E(x_0, x_{ij}) = x_{i(j+1)} \in \mathcal{C} = \mathcal{K}$

x_{10}	$\xrightarrow{f}$	x_{11}	$\xrightarrow{f}$	x_{12}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{1t}
x_{20}	$\xrightarrow{f}$	x_{21}	$\xrightarrow{f}$	x_{22}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{2t}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_{m0}	$\xrightarrow{f}$	x_{m1}	$\xrightarrow{f}$	x_{m2}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{mt}



www.isea.gov.in

Collision Resistance

- For $i = 1$ to m choose x_{i0} at random. For $j = 0$ to $(t-1)$ evaluate $f(x_{ij}) = x_{i(j+1)}$. The table will “cover” mt keys.
- The probability of obtaining a collision is negligible if $(mt)t = mt^2 \leq N$. For details we refer to Hellman and Biryukov-Shamir.

x_{10}	$\xrightarrow{f}$	x_{11}	$\xrightarrow{f}$	x_{12}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{1t}
x_{20}	$\xrightarrow{f}$	x_{21}	$\xrightarrow{f}$	x_{22}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{2t}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_{m0}	$\xrightarrow{f}$	x_{m1}	$\xrightarrow{f}$	x_{m2}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{mt}

1. Hellman, M.E.: A cryptanalytic time-memory trade-off. IEEE Trans. Information Theory 26(4), 401--406 (1980)
2. Biryukov, A., Shamir, A.: Cryptanalytic time/memory/data tradeoffs for stream ciphers. ASIACRYPT 2000. pp. 1--13.

TMTO matrix

- All these mt vectors successively generated by f starting from randomly chosen vectors x_{i0} , $1 \leq i \leq m$, can be summarized in a single $m \times t$ matrix called TMTO-matrix whose elements are called keys.
- TMTO matrix

$$\begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1t} \\ x_{21} & x_{22} & \cdots & x_{2t} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mt} \end{bmatrix}$$

Week 4

Module 4

- Time-memory tradeoff (TMTO) attack
- Hellman's TMTO curve

Collision Resistance

- For $i = 1$ to m choose x_{i0} at random. For $j = 0$ to $(t-1)$ evaluate $f(x_{ij}) = x_{i(j+1)}$. The table will “cover” mt keys.
- The probability of obtaining a collision is negligible if $(mt)t = mt^2 \leq N$. For details we refer to Hellman and Biryukov-Shamir.

x_{10}	$\xrightarrow{f}$	x_{11}	$\xrightarrow{f}$	x_{12}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{1t}
x_{20}	$\xrightarrow{f}$	x_{21}	$\xrightarrow{f}$	x_{22}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{2t}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_{m0}	$\xrightarrow{f}$	x_{m1}	$\xrightarrow{f}$	x_{m2}	$\xrightarrow{f}$	$\dots$	$\xrightarrow{f}$	x_{mt}

1. Hellman, M.E.: A cryptanalytic time-memory trade-off. IEEE Trans. Information Theory 26(4), 401--406 (1980)
2. Biryukov, A., Shamir, A.: Cryptanalytic time/memory/data tradeoffs for stream ciphers. ASIACRYPT 2000. pp. 1--13.

TMTO matrix

- All these mt vectors successively generated by f starting from randomly chosen vectors x_{i0} , $1 \leq i \leq m$, can be summarized in a single $m \times t$ matrix called TMTO-matrix whose elements are called keys.
- TMTO matrix

$$\begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1t} \\ x_{21} & x_{22} & \cdots & x_{2t} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mt} \end{bmatrix}$$

How to cover all the keys?

- Each TMTO-matrix can store at most mt keys where $mt^2 = N$ without the risk of having to store the same key at several positions.
- Instead of f we use $h_j \circ f(x) = h_j(f(x))$, for all $x \in \mathbb{F}_2^n$ where h_i permutes the coordinates of the input vector. Let h_j for $j = 1, \dots, t$ be t such distinct functions. h_1 is the identity permutation.

How to cover all the keys?

- Let the TMTO-matrix generated by $h_j \circ f$, j th TMTO-matrix, be denoted by

$$\begin{bmatrix} x_{11}^{(j)} & x_{12}^{(j)} & \dots & x_{1t}^{(j)} \\ x_{21}^{(j)} & x_{22}^{(j)} & \dots & x_{2t}^{(j)} \\ \vdots & \vdots & \vdots & \vdots \\ x_{m1}^{(j)} & x_{m2}^{(j)} & \dots & x_{mt}^{(j)} \end{bmatrix}$$

How to cover all the keys?

- Hellman envisioned that if each of these functions is used to generate a TMTO-matrix, the resulting t tables will cover all the $mt^2 = N$ keys, “defying” the Birthday Paradox.
- Hellman was proved to be correct. The time complexity of the preprocessing phase is $P = N$, since all the keys have to be visited.
- This computation may take several weeks or even year! However, this is a one-time investment.

Memory requirement to store the tables

- Instead of storing all the mt keys of each of the t matrices, only the starting and the ending keys are stored in a list.
- Instead of the j -th TMT0-matrix we construct a list L_j of the pairs $\left(x_{i0}^{(j)}, x_{it}^{(j)}\right)$ for $i = 1, \dots, m$, sorted with respect to the second-component.
- Let us denote the list corresponding to the j th matrix by L_j .
- Therefore, we need mt memory locations to store t matrices.

TMTO Lists

- The TMTO lists are as follows:

$$L_1 = \begin{bmatrix} x_{11}^{(1)} & \cdots & x_{1t}^{(1)} \\ x_{21}^{(1)} & \cdots & x_{2t}^{(1)} \\ \vdots & \vdots & \vdots \\ x_{m1}^{(1)} & \cdots & x_{mt}^{(1)} \end{bmatrix} \quad L_2 = \begin{bmatrix} x_{11}^{(2)} & \cdots & x_{1t}^{(2)} \\ x_{21}^{(2)} & \cdots & x_{2t}^{(2)} \\ \vdots & \vdots & \vdots \\ x_{m1}^{(2)} & \cdots & x_{mt}^{(2)} \end{bmatrix}$$

$$\cdots \quad L_t = \begin{bmatrix} x_{11}^{(t)} & \cdots & x_{1t}^{(t)} \\ x_{21}^{(t)} & \cdots & x_{2t}^{(t)} \\ \vdots & \vdots & \vdots \\ x_{m1}^{(t)} & \cdots & x_{mt}^{(t)} \end{bmatrix}$$

Online Phase: Algorithm

Input: (x_0, y_0) , L_j for $j = 1, \dots, t$;

Output: The secret key k_0 ;

for $i = 0$ **to** $t - 1$ **do**

for $j = 1$ **to** t **do**

$y' \leftarrow h_j(y_0)$;

$y'_i \leftarrow (h_j \circ f)^i(y')$;

 ▷ search in the sorted list L_j , for all j

if $y'_i \in L_j$ **then**

 determine ℓ such that $y'_i = x_{\ell t}^{(j)}$;

return $k = (h_j \circ f)^{(t-i-1)}(x_{\ell 0}^{(j)})$;

end

end

end

Analysis

- In case all the keys are covered in the tables, the attack will be successful.
- The memory requirement for each table is m . So, for t tables the memory requirement is $M = mt$.
- If we assume that the tables are sorted, the time to search each table is $\log_2 m$.
- In case we have to search all t tables before finding a match, the time complexity is $t \times \log_2 m$.

Analysis

- To search in a table in the i -th iteration, first f has to be applied $(i-1)$ times on the ciphertext. If there is a match, f has to be applied on the corresponding starting point (SP) $(t-i)$ times.
- The total number of times f has to be applied is
$$t - i + i - 1 = t - 1.$$
- The time complexity of the attack is $T = (t-1) \times t \times \log_2 m$.

Complexity of Attack

- M = the storage memory.
- For each table we need m locations to store the first and the last points of each row.
- For t tables we require mt locations.
- The preprocessing and online space complexity is $M = mt$.

Preprocessing Time Complexity



- While preparing the tables we have to cover all the N keys.
- The preprocessing time complexity is $P = N$.

Online Phase Time Complexity

- The function f is applied on the output t times for each table to determine whether the key appears in the table.
- Since there are t tables and for each table computation is of the order t we have the online phase time complexity $T = t^2$.
- In case we include the time complexity of the table search in the online phase $T = t^2 \log_2 m$.

The Trade-Off Curve

- The previous analysis gives us the following equations:
 - $M = mt$
 - $P = N$
 - $N = mt^2$
 - $T = t^2$
 - $TM^2 = t^2 m^2 t^2 = (mt^2)^2 = N^2$
- The final expression
$$TM^2 = N^2$$
is known as the Hellman's time-memory tradeoff curve.

End of Week 4: Module 4